Namma Santhe Ledger

My Market Ledger

Android App Development Project Report

Small Business Finance Management – Android Application

Project Title	Andriod App Development using GenAI-Namma-Santhe Ledger (Finance) – Smart Business Ledger App
Project Number	56
Platform	Android (Kotlin + XML Layouts)
Domain	Small Business / Finance Management / Commerce
Category	Android Application with Room Database & Work Manager
Report Type	Internship / Academic Project Report

Prepared by:
Jyothishree D
USN: 1ST22IS022
Sambhram Institute of Technology
Department of Information Science Engineering
2025 – 2026

1. Introduction

Namma Santhe Ledger is an Android application developed to help small-scale merchants, vendors, and shopkeepers in India's local markets (santhe) digitally manage their credit transactions and customer dues. The application name combines Kannada and English, reflecting its core purpose: to be the trusted digital ledger for every local market seller.

Developed using Kotlin with XML-based layouts, the app integrates Room Database for permanent local storage, WorkManager for background task scheduling, and Android's SmsManager for automated customer payment reminders. The application targets the informal retail economy — a segment that has historically relied on handwritten ledgers, notebooks, and memory to track credit sales.

The application addresses three critical needs of small market vendors: recording transactions against customer names and phone numbers, tracking the total outstanding amount at a glance, and sending automated weekly SMS reminders to customers who have pending dues. A PINbased login system ensures that sensitive financial data remains secure even on shared devices. A shop profile screen allows each vendor to personalise the app with their business identity, which is referenced in automated SMS messages sent to customers.

Namma Santhe Ledger is designed to function entirely offline, requiring no internet connection for its core operations. Data is stored persistently on-device using Room Database, ensuring that transaction records survive app restarts, device reboots, and long periods of inactivity. The app targets the vast and underserved community of informal vendors across Karnataka and rural India who lack access to point-of-sale systems or accounting software.

2. Problem Statement

India's informal retail economy is one of the largest in the world, comprising millions of market vendors, street sellers, and small shopkeepers who extend credit to regular customers as a matter of routine. This informal credit system, while socially valuable, presents significant operational challenges for vendors who lack formal accounting tools. Several systemic problems affect this segment:

Challenge	Description
Paper Ledger Loss	Handwritten notebooks are easily lost, damaged by water or fire, or become illegible over time, causing permanent loss of credit records.
Memory Dependency	Vendors often rely on memory alone to track dues, resulting in errors, disputes with customers, and financial losses.
No Payment Follow-up	Without automated reminders, vendors must personally and repeatedly contact customers to collect dues, which is time-consuming and socially awkward.
No Financial Overview	Vendors have no quick way to view their total outstanding amount or number of credit transactions at a glance, making financial planning difficult.
Data Insecurity	Shared physical registers allow anyone to view, modify, or remove financial records. There is no access control or authentication on traditional ledgers.

These interconnected challenges highlight the urgent need for a dedicated, offline-first digital ledger application that vendors can use with minimal technical knowledge, on low-cost Android smartphones, without internet connectivity.

3. Project Vision

Namma Santhe Ledger envisions a world where every local market vendor — regardless of education level, technical background, or economic status — has access to a reliable, secure, and intelligent financial management tool in their own language.

The project is built on four foundational pillars:

- Accessibility – Financial management tools must be available to every vendor, regardless of smartphone model, internet availability, or digital literacy. The app is designed for minimum API 21 devices and requires no internet for core features.
- Simplicity – The interface must be usable by vendors who may have limited literacy and no prior experience with accounting software. Large touch targets, clear iconography, and a minimal learning curve are non-negotiable design principles.
- Reliability – Transaction data must never be lost. Room Database provides ACIDcompliant local storage that survives app crashes, device reboots, and accidental app closures.
- Automation – The burden of payment collection must be reduced through technology. Weekly SMS reminders automatically notify customers of their outstanding dues, removing the social friction of manual follow-up.

Namma Santhe Ledger is not merely a note-taking app; it is a financial empowerment tool designed to bring the discipline of formal accounting to the informal economy, one transaction at a time.

4. Scope of the Application

4.1 Target Users

- Small-scale market vendors and santhe (weekly market) sellers across Karnataka
- Kirana store owners and general merchants who extend daily credit
- Street food vendors, vegetable sellers, and mobile traders
- Any small business owner who records transactions manually in a register

4.2 Platform

- Android smartphones with minimum API Level 21 (Android 5.0 Lollipop and above)
- Optimised for low-end and mid-range devices with limited RAM
- APK size kept minimal to support devices with limited internal storage

4.3 Offline Functionality

- All transaction data is stored locally using Room Database — no cloud or server required
- Core features including recording, viewing, and deleting transactions work fully offline
- SMS reminders use the device's native SmsManager, requiring only mobile network access (not internet)

4.4 Key Operational Scenarios

- Recording a new credit transaction against a customer name, phone number, and amount
- Viewing a real-time dashboard showing total transactions and outstanding amount
- Sending automated weekly SMS reminders to all customers with recorded phone numbers
- Securely logging in and out of the app using a 4-digit PIN
- Managing shop profile information which appears in outgoing SMS reminders

5. Objectives

The project is aligned with the following measurable objectives:

- Provide a fast and accurate transaction recording system — Ensure that a new transaction can be saved within one screen interaction from the home dashboard, with fields for customer name, phone number, and amount.
- Deliver a real-time financial overview — Maintain a live summary showing total number of recorded transactions and the total outstanding amount, updated instantly with every addition or deletion.
- Ensure permanent, offline data persistence — All transaction records must survive application restarts, device reboots, and extended periods of inactivity using Room Database.
- Automate customer payment reminders — Enable weekly automated SMS messages to be sent to all customers whose phone numbers are recorded, containing their outstanding balance and the vendor's shop name.
- Protect sensitive financial data — Implement PIN-based login so that only the authorised vendor can access transaction records, with a change PIN option available in settings.
- Provide a professional vendor identity — Allow vendors to save their shop name, owner name, phone, and address in a profile screen, with the shop name appearing in all outgoing SMS reminders.
- Support daily reminder notifications — Allow vendors to set a daily push notification reminder at a chosen time to encourage regular transaction recording.

6. Features

6.1 Core Features

Feature	Description
PIN-Based Login	4-digit PIN authentication screen shown at app launch. First-time users create a PIN; returning users enter their saved PIN. Supports change PIN via Settings.
Professional Dashboard	Home screen displays total transaction count and total outstanding amount in real-time summary cards, alongside the dated transaction entry form.
Transaction Recording	Capture customer name, phone number, and amount. Transactions are stored in Room Database with an auto-generated timestamp. Newest entries appear at the top.
Persistent Storage	Room Database (SQLite) stores all transactions permanently ondevice. Data is retained after app closure, device restart, or extended inactivity.
Transaction Management	Long-press any transaction to delete it with a confirmation dialog. A Clear All option with confirmation wipes all records when needed.
Weekly SMS Reminders	WorkManager schedules a background task every 7 days. For each customer with a recorded phone number, an SMS is sent showing their outstanding balance and the vendor's shop name.
Daily Push Notification	Vendors set a daily reminder time via a time picker in Settings. AlarmManager triggers a push notification at the chosen time every day.
Shop Profile	Vendors save their shop name, owner name, phone number, and address. The shop name is displayed in the main header and included in outgoing SMS reminders.
Settings Screen	Centralised settings for toggling daily reminders, setting reminder time, enabling or disabling weekly SMS, changing PIN, and managing security preferences.
Logout	Vendors can securely log out from the 3-dot menu with a confirmation dialog. Session state is cleared and the login screen is shown on next launch.

6.2 Future Scope Features

- UPI Payment Integration – Allow vendors to share UPI payment links directly with customers via SMS or WhatsApp.
- PDF Statement Export – Generate a formatted PDF statement of all transactions for a customer or date range.
- Multi-Language Support – Extend the interface to support Kannada, Hindi, Tamil, and Telugu in addition to English.
- Cloud Backup – Optional encrypted cloud sync via Firebase to protect data against device loss.
- Customer-wise View – Group and filter transactions by individual customer to see per-customer outstanding balances.
- Biometric Authentication – Replace PIN login with fingerprint authentication on supported devices.

7. Functional Requirements

The following functional requirements define the core behaviours the system must support:

FR-1: Authentication

- The system shall display a PIN entry screen on every app launch.
- First-time users shall be prompted to create a 4-digit PIN which is stored in SharedPreferences.
- Subsequent users shall be required to enter their saved PIN to access the dashboard.
- An incorrect PIN entry shall display an error message and clear the input field.

FR-2: Transaction Recording

- The system shall provide input fields for customer name, phone number (optional), and amount.
- The system shall validate that name and amount fields are not empty before saving.
- The system shall validate that the amount is a positive numeric value.
- On successful save, the system shall store the transaction in Room Database with a formatted timestamp and refresh the transaction list and summary cards.

FR-3: Dashboard Summary

- The system shall display the total count of all recorded transactions in real time.
- The system shall display the sum of all transaction amounts, formatted with the Indian Rupee symbol (₹).
- The system shall update both summary values immediately after any addition or deletion.

FR-4: Transaction Management

- The user shall be able to delete any individual transaction by long-pressing it.
- The system shall display a confirmation dialog before executing any deletion.
- The user shall be able to clear all transactions via a Clear All option with a confirmation dialog.

FR-5: Weekly SMS Reminders

- The system shall use WorkManager to schedule a repeating background task at 7-day intervals.

- The task shall query all transactions grouped by customer name.

- For each customer with a non-empty phone number, the system shall compose and send an SMS containing the customer name, shop name, and total outstanding amount.
- The SMS shall be sent using the device's native SmsManager without requiring internet access.

FR-6: Daily Reminder Notification

- The user shall be able to set a daily reminder time using a time picker dialog in Settings.
- The system shall use AlarmManager to trigger a push notification at the chosen time every day.
- The notification shall contain the shop name and a prompt to record today's transactions.
- The user shall be able to toggle the daily reminder on or off independently.

FR-7: Profile Management

- The user shall be able to enter and save shop name, owner name, phone number, and address.
- The shop name shall be displayed in the main header of the dashboard.
- Profile data shall be stored in SharedPreferences and persist across sessions.

FR-8: Settings & Security

- The user shall be able to change their PIN from the Settings screen.
- The user shall be able to toggle and configure daily and weekly reminders from Settings.
- The user shall be able to log out from the main menu with a confirmation dialog.

8. Non-Functional Requirements

Category	Requirement	Acceptance Criterion
Performance	Load Time	Dashboard must fully render within 800 ms of login on a mid-range Android device.
Performance	Transaction Save	A new transaction must be saved and reflected in the list within 500 ms of tapping the Save button.
Usability	UI Simplicity	Any user must be able to record a transaction within 3 taps from the login screen.
Usability	Legibility	All primary text must use a minimum font size of 13sp. Summary card values must use a minimum of 24sp.

Reliability	Data Persistence	100% of transaction records must be retained after app closure, device restart, and 30+ days of inactivity.
Reliability	Crash Rate	Application crash rate must not exceed 1% during standard transaction recording and deletion flows.
Accessibility	Touch Targets	All interactive elements must have a minimum touch target size of 48x48dp per Material Design guidelines.
Compatibility	Android Versions	Application must support Android 5.0 (API 21) through Android 14 (API 34).
Security	Data Privacy	All transaction data remains on-device. No data is transmitted to external servers. PIN is stored locally in SharedPreferences.
Offline Support	Core Features	All core features (recording, viewing, deleting, notifications) must function without any network connection.

9. System Architecture

Namma Santhe Ledger follows a single-tier, client-only architecture. All data processing, storage, and business logic occur entirely on-device. There are no backend servers, cloud databases, or API dependencies required for the application's core operation. **9.1 Technology Stack**

Layer	Technology	Purpose
Language	Kotlin	Primary development language; concise, null-safe, fully interoperable with Android SDK.
UI Framework	XML Layouts + AppCompat	Traditional View-based UI with LinearLayout, ListView, and Material components for broad device compatibility.
Local Database	Room Database (SQLite)	ACID-compliant ORM over SQLite. Provides permanent on-device storage with DAO-based queries and migration support.
Background Tasks	WorkManager	Schedules and executes weekly SMS reminder tasks reliably, surviving device reboots and app closures.

Notifications	AlarmManager + NotificationManager	Triggers daily reminder notifications at vendor-specified times. Supports Android 8+ notification channels.
SMS Engine	Android SmsManager	Native SMS dispatch without internet. Sends customer payment reminders directly from the device's SIM.
Session Storage	SharedPreferences	Stores PIN, login state, profile data, and reminder settings as lightweight keyvalue pairs.
Build System	Gradle (Groovy DSL)	Dependency management, build configuration, and APK generation including kotlin-kapt for Room annotation processing.
Min. SDK	API Level 21	Targets approximately 98% of active Android devices globally, including low-cost smartphones prevalent in rural India.

9.2 Architectural Overview

The application follows a simplified MVC (Model-View-Controller) pattern appropriate for a single-activity, XML-layout Android app:

- Model Layer – The data package contains the Room @Entity (Transaction), @Dao (TransactionDao), and @Database (AppDatabase) classes. This layer owns all data access logic and migration handling.
- View Layer – XML layout files define the UI structure (activity_main.xml, activity_login.xml, activity_profile.xml, activity_settings.xml, list_item_transaction.xml). Custom drawables (card_background, avatar_background, input_background, bell_background, pin_input_background) provide visual styling.
- Controller Layer – Activity classes (MainActivity, LoginActivity, ProfileActivity, SettingsActivity) orchestrate user interaction, invoke database operations on background threads via Executors, and update the UI on the main thread via runOnUiThread.
- Background Workers – ReminderReceiver (BroadcastReceiver) handles daily notification delivery. WeeklyReminderWorker (Worker) performs database queries and SMS dispatch on a background thread managed by WorkManager.

9.3 Database Design

The Room database contains a single table — transactions — with the following schema:

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY, AUTO	Auto-generated unique identifier for each transaction.
customerName	TEXT	NOT NULL	Name of the customer who owes the amount.
phoneNumber	TEXT	DEFAULT ''	Customer phone number for SMS reminders. Optional field.
amount	REAL	NOT NULL	Credit amount in Indian Rupees (stored as Double).
timestamp	TEXT	NOT NULL	Formatted date and time of transaction recording.

10. User Flow

The application is designed to minimise friction in high-frequency daily use scenarios. The complete user journey from app launch to transaction recording is described below:

#	Screen	User Action & System Response
1	**Login Screen**	App launches. If first use, vendor creates a 4-digit PIN. On subsequent launches, vendor enters their PIN to authenticate.
2	**Dashboard**	Vendor sees the main dashboard with summary cards showing total transactions and total outstanding amount. Today's date appears in the header with the shop name.
3	**Add Transaction**	Vendor fills in customer name, phone number (optional), and amount. Taps Save Transaction. The transaction is saved to Room Database and appears at the top of the list. Summary cards update instantly.
4	**View History**	The transaction list shows all records with customer avatar initials, name, phone number, timestamp, and amount. Newest entries appear first.
5	**Delete Entry**	Vendor long-presses a transaction. A confirmation dialog appears. On confirmation, the record is deleted from Room Database and the list and summaries update.

6	**Profile**	Vendor taps 3-dot menu → My Profile. Enters shop name, owner name, phone, and address. Saves profile. Shop name reflects in the dashboard header immediately on return.
7	**Settings**	Vendor taps 3-dot menu → Settings. Enables daily reminder and picks time. Enables weekly SMS reminders. Changes PIN if needed.
8	**Weekly SMS**	WorkManager fires every 7 days. The app queries all customers with phone numbers, computes their total dues, and sends each an SMS. No user action required.
9	**Daily Reminder**	At the vendor's chosen time, a push notification appears: 'Don't forget to record today's transactions!' Tapping opens the dashboard.
10	**Logout**	Vendor taps 3-dot menu → Logout. Confirmation dialog shown. On confirm, session is cleared and login screen is presented.

11. Technical Implementation

11.1 Room Database & Migration

Transaction records are stored using the Room persistence library, which provides a type-safe abstraction over SQLite. The TransactionDao interface exposes four operations: getAll() (ordered by id DESC to show newest first), insert(), delete(), and deleteAll(). The AppDatabase singleton uses a double-checked locking pattern to ensure a single database instance throughout the app lifecycle.

A database migration (version 1 → version 2) handles the addition of the phoneNumber column using an ALTER TABLE statement, ensuring that existing users who upgrade from an earlier version do not lose their transaction history. The migration is registered with addMigrations() on the RoomDatabase.Builder.

11.2 Background Threading

Room does not permit database operations on the main (UI) thread. All database calls are executed using a single-threaded Executors.newSingleThreadExecutor() instance. Results are posted back to the UI thread via runOnUiThread{}, which updates the RecyclerView/ListView adapter and summary TextViews. This pattern avoids the need for LiveData or coroutines while maintaining thread safety.

11.3 WorkManager for Weekly SMS

WeeklyReminderWorker extends Worker and implements doWork() to perform the weekly customer SMS dispatch. It is enqueued as a PeriodicWorkRequest with a 7-day interval using

WorkManager.enqueueUniquePeriodicWork() with ExistingPeriodicWorkPolicy.REPLACE, ensuring that enabling the feature always resets the 7-day cycle. The worker queries the database for all transactions, groups them by customerName, computes the total per customer, and sends an SMS to each customer with a non-empty phone number.

11.4 SMS Dispatch

SMS messages are sent using Android's SmsManager API, which dispatches messages via the device's active SIM card without requiring internet. The implementation handles the API level differences: on Android 12 (API 31) and above, SmsManager is obtained via context.getSystemService(SmsManager::class.java); on earlier versions, SmsManager.getDefault() is used. Each SMS is personalised with the customer's name, total outstanding amount, and the vendor's shop name from SharedPreferences.

11.5 AlarmManager for Daily Reminders

The daily notification reminder uses AlarmManager.setRepeating() with AlarmManager.RTC_WAKEUP and an interval of AlarmManager.INTERVAL_DAY. The alarm triggers ReminderReceiver, a BroadcastReceiver that builds and displays a NotificationCompat notification. On Android 8.0 and above, a notification channel is created before posting the notification. The PendingIntent includes FLAG_IMMUTABLE on API 23+ for security compliance.

11.6 Custom ListView Adapter

TransactionAdapter extends ArrayAdapter<Transaction> and inflates the custom list_item_transaction.xml layout for each entry. The getView() method populates the avatar initial (first character of customer name, uppercased), customer name, phone number, formatted timestamp, and amount. The adapter is backed directly by the mutable transactions list in MainActivity, and notifyDataSetChanged() is called after every database operation to refresh the display.

11.7 Security – PIN Authentication

The 4-digit PIN is stored in SharedPreferences under a named preferences file (LoginPrefs). The login session state (isLoggedIn boolean) is set to true on successful login and cleared to false on logout. LoginActivity checks this flag on creation and routes directly to MainActivity if the session is active, avoiding repeated PIN entry during a single session. PIN hashing was not implemented in version 1.0 but is planned as a security enhancement in a future version.

12. Project File Structure

File / Directory	Purpose
app/build.gradle	App-level Gradle config: plugins, SDK versions, Room, WorkManager, and kapt dependencies.
data/Transaction.kt	Room @Entity data class representing a single credit transaction.
data/TransactionDao.kt	Room @Dao interface with getAll, insert, delete, deleteAll, and getCustomersWithPhone queries.
data/AppDatabase.kt	Room @Database singleton with version management and migration definitions.
LoginActivity.kt	PIN creation and authentication screen. Routes to MainActivity on success.
MainActivity.kt	Main dashboard: transaction list, summary cards, add/delete logic, options menu.
ProfileActivity.kt	Shop profile form: name, owner, phone, address. Persists via SharedPreferences.
SettingsActivity.kt	Settings for daily/weekly reminders, reminder time picker, and change PIN.
ReminderReceiver.kt	BroadcastReceiver that builds and posts daily reminder push notifications.
WeeklyReminderWorker.kt	WorkManager Worker that dispatches weekly SMS reminders to all customers.
TransactionAdapter.kt	Custom ArrayAdapter inflating list_item_transaction.xml for each transaction.
res/layout/activity_main.xml	Dashboard layout: header, summary cards, add form, transaction ListView.
res/layout/activity_login.xml	Login screen layout with PIN entry and login/create button.
res/layout/activity_profile.xml	Profile form layout with ScrollView for shop details.
res/layout/activity_settings.xml	Settings layout with switches, time picker button, and change PIN button.

res/layout/list_item_transaction.xml	Custom list row: avatar circle, customer info, timestamp, amount.
res/drawable/*.xml	Shape drawables for card backgrounds, input fields, avatar circles, and bell icon.
res/menu/main_menu.xml	3-dot overflow menu items: Profile, Settings, Logout.
AndroidManifest.xml	App manifest: permissions, activity declarations, receiver registration, launcher activity.

13. Permissions & Privacy

Permission	Required For	Notes
SEND_SMS	Weekly customer reminders	Runtime permission required on Android 6.0+. User must grant before SMS features activate.
POST_NOTIFICATIONS	Daily reminder notifications	Required on Android 13+. User prompted on first notification trigger.
SCHEDULE_EXACT_ALARM	Daily reminder at exact time	Required on Android 12+. User may need to grant via system settings.
RECEIVE_BOOT_COMPLETED	Restart alarms after reboot	Ensures daily reminders survive device restarts.

No personal user data is transmitted to any external server. All transaction records, profile information, and PIN data remain exclusively on the vendor's device. The application does not collect analytics, crash reports, or usage statistics. Customer phone numbers are used solely for the purpose of sending payment reminder SMS messages at the vendor's request.

14. Impact Goals

14.1 Economic Empowerment

By digitising the informal credit ledger, Namma Santhe Ledger directly reduces financial losses caused by forgotten dues, illegible records, or misplaced notebooks. Vendors who consistently track and recover credit transactions improve their working capital and reduce their dependence on informal moneylenders to bridge cash flow gaps.

14.2 Digital Inclusion

The application demonstrates that effective financial management software does not require a smartphone with high-end specifications, a reliable data connection, or significant digital literacy. By targeting API 21 and above and operating fully offline, the app is accessible to vendors using affordable handsets across rural Karnataka and India.

14.3 Reduction of Business Disputes

Automated SMS reminders sent with a specific outstanding amount and shop name provide customers with a clear, documented reference for their dues, reducing the ambiguity and social friction that often accompanies informal credit disputes. A written (digital) record is more defensible than verbal recollection.

14.4 Promotion of Financial Discipline

Daily reminder notifications encourage vendors to record transactions at the end of each market day rather than relying on memory. This habit formation improves the completeness and accuracy of financial records over time, providing vendors with a more reliable picture of their business performance.

14.5 Scalability of Impact

The modular architecture of Namma Santhe Ledger — with separate data, activity, and worker layers — supports rapid expansion. Future versions can add multi-language support, cloud backup, customer-wise filtering, and UPI integration without requiring a fundamental redesign, ensuring that the application can grow alongside the needs of its users.

15. Success Criteria

The project will be considered successfully completed when the following criteria are met:

#	Success Criterion	Measurement	Priority
1	App loads and reaches the Dashboard within 800 ms on a reference device (Redmi 9A or equivalent).	Android Profiler measurement	Critical

2	A new transaction can be recorded within 3 taps from the login screen.	Manual UI walkthrough	Critical
3	All transaction records are retained after device reboot and 30+ days of inactivity.	Room DB inspection test	Critical
4	PIN login correctly blocks access with wrong PIN and grants access with correct PIN.	Manual security testing	Critical
5	Weekly SMS is sent to all customers with recorded phone numbers, containing correct name, amount, and shop name.	SMS log verification	Critical
6	Daily notification appears at the set time with correct content.	Manual notification test	High
7	Summary cards update correctly (count and total) after every add and delete operation.	Manual validation	High
8	App does not crash during complete walkthrough of all screens and features.	Manual regression testing	High
9	Profile shop name appears correctly in the dashboard header and in outgoing SMS messages.	Manual UI and SMS check	Medium
10	All features work fully in Airplane Mode (except SMS which requires mobile network).	Airplane Mode test on device	Medium

Conclusion

Namma Santhe Ledger represents a focused and socially meaningful application of Android development in the domain of informal small business finance management. By addressing a clear and well-defined problem — the lack of a reliable, offline, and intelligent digital ledger for India's market vendors — the project delivers measurable practical value alongside technical merit.

The application's architecture prioritises reliability, simplicity, and offline-first operation above all else. Every design decision, from Room Database for permanent storage to WorkManager for resilient background task scheduling and SmsManager for internet-free customer communication, is motivated by a single question: will this work for a vendor standing in a busy market with a basic smartphone and no internet connection?

The project also establishes a strong technical foundation for future expansion. The separation of data, view, and controller layers supports the addition of new features — cloud backup, multi-language support, UPI integration, biometric authentication — without requiring a fundamental redesign of the existing codebase. The Room migration framework ensures that future schema changes can be deployed to existing users without data loss.

In conclusion, Namma Santhe Ledger is not simply a technical deliverable — it is a small but meaningful step toward financial inclusion and digital empowerment for the millions of informal vendors who form the backbone of India's local economy.